



Byte Capsule

STAY SAFE, STAY SECURE

Web Application Security - EHP

Date: 5-12-2025

Name: Md Mazad Ul Islam

Email: mdmajedulislam4111999@gmail.com

Contact Number: +8801746777198

Address: Rangpur, Dhaka, Bangladesh

Table of contents:

Confidentiality Statement.....	3
Executive Summary.....	3
• Purpose of the Assessment.....	3
• Assessment Overview.....	3
• High-Level Risk Rating.....	3
Critical Findings Summary.....	4
Scope and Out-of-Scope.....	4
• In-Scope Assets.....	4
• Out-of-Scope Components.....	4
Engagement Details / Assessment Guidelines.....	4
Methodology.....	5
Environment Details.....	6
• Target URLs.....	6
• Api End point.....	6
• Technology Stack.....	6
• Testing Environment.....	7
Steps of Finding Bugs	8
Results Overview.....	8
Findings Summary Table.....	10
Detailed Vulnerability Reports.....	11
1. MFA Issues – OTP Redirect Manipulation.....	11
2. OTP Validation Bypass.....	12
3. IDOR – Customer Details Exposure.....	13
4. Business Logic Flaw – Price Manipulation.....	16
5. Email Spoofing (SPF, DKIM, DMARC Weaknesses)	18
6. Self-XSS.....	22
7. Clickjacking Vulnerability.....	23
8. Lack of Secure & HttpOnly Cookies.....	24
9. Text Injection (Content Injection)	26
10. Missing Security Headers.....	28
11. TLS/SSL Misconfigurations.....	31
12. Fingerprinting & Information Disclosure.....	34
Recommendations (Global Fixes)	36
Conclusion.....	37

Confidentiality Statement

This document contains confidential and sensitive information pertaining to the security posture, vulnerabilities, and technical details of the target system. It is intended solely for the authorized recipients of **Sheba.xyz** and its designated security and management personnel.

Unauthorized access, disclosure, distribution, or reproduction of this report, in whole or in part, is strictly prohibited. The information contained herein must be protected in accordance with the organization's data handling and information security policies.

All findings, evidence, and recommendations within this report are provided exclusively for the purpose of improving the security of the tested systems. Any misuse of the information for activities other than authorized remediation is illegal and may result in legal action.

Executive Summary:

• Purpose of the Assessment:

The purpose of this assessment was to evaluate the security posture of **Sheba.xyz** and its associated systems by identifying vulnerabilities that could lead to unauthorized access, data exposure, financial manipulation, or service disruption. The test was performed to help the organization understand existing weaknesses, validate the effectiveness of current security controls, and provide actionable recommendations to improve overall resilience against real-world cyber threats.

• Assessment Overview:

A comprehensive penetration test was conducted following an attacker-oriented methodology that included reconnaissance, enumeration, vulnerability discovery, exploitation, and post-exploitation validation. The assessment covered both web application and API endpoints, including authentication workflows, user data handling, session management, and server-side configurations.

During the engagement, multiple critical and high-severity vulnerabilities were discovered—most notably **OTP validation bypass**, **IDOR-based data exposure**, and **business logic manipulation**, each of which poses a significant risk to user accounts and organizational integrity. Additional medium and low-severity issues were found in areas such as **security headers**, **cookie configurations**, **email spoofing protections**, and **information disclosure**.

• High-Level Risk Rating:

Based on the vulnerabilities identified and their potential impact on confidentiality, integrity, and availability, the overall security risk rating for the target environment is:

Overall Risk Level: HIGH

This rating is driven primarily by the presence of several **critical vulnerabilities** that can enable:

- Full account takeover
- Unauthorized access to customer data
- Financial fraud through price manipulation
- Phishing attacks through email spoofing
- Increased exploitability due to missing or weak security controls

Immediate remediation of the critical and high-severity findings is strongly recommended to reduce the likelihood of real-world exploitation.

Critical Findings Summary:

The assessment of **Sheba.xyz** identified several **critical vulnerabilities** that present a high risk to the organization's security, user data, and business operations. These issues require immediate remediation to prevent potential exploitation.

Key Critical Findings

1. **OTP Validation Bypass**
 - **Impact:** Allows unauthorized users to log in without completing OTP verification.
 - **Risk:** Full account takeover, leading to exposure of sensitive user data.
2. **IDOR – Customer Details Exposure**
 - **Impact:** Attackers can access other users' personal and financial information by manipulating user identifiers.
 - **Risk:** Data privacy violations, regulatory non-compliance, and reputational damage.
3. **Business Logic Flaw – Price & Quantity Manipulation**
 - **Impact:** Enables unauthorized modification of product pricing and quantities.
 - **Risk:** Direct financial loss and fraudulent transactions.

Overall Risk:

These critical issues pose a **high risk** to both **user security** and **organizational operations**. Immediate remediation and verification are strongly recommended to mitigate exploitation.

Scope and Out of Scope:

In-Scope Assets:

MFA issues, WAF bypass, Open redirects, Internal IP disclosure, Accessible non-sensitive files/directories (README.TXT, robots.txt, .gitignore, etc.), Self XSS, Text injection, Email spoofing (SPF, DKIM, DMARC, From: spoofing), Descriptive error messages, Fingerprinting/Banner disclosure, Clickjacking, CSRF issues (non-critical), Missing Secure & HttpOnly cookie flags, Lack of rate limiting, Login/Forgot Password brute force or weak passwords, HTTPS mixed content, Username/email enumeration, Missing HTTP security headers, TLS/SSL issues, Out-of-date software, Use of known-vulnerable components with proof.

Out-of-Scope Components:

DDoS or defacement, social engineering/phishing, Physical attacks on facilities, Relay/RollJam attacks on keyfob/NFC/phone-as-key, Bug disclosure before author permission.

Engagement Details / Assessment Guidelines:

Assessment Type:

- White Box

Assessment Duration:

- Start Date: 15/11/2025
- End Date: 05/12/2025

Assessment :

- Prepared By: Md Mazad Ul Islam

Rules of Engagement (ROE):

- Testing limited to in-scope systems only.

- No Denial-of-Service (DoS) or destructive attacks allowed.
- No brute-force attempts beyond agreed thresholds.
- Only authorized endpoints and domains were tested.
- Test data modification performed only with controlled test payloads.

Allowed Testing Techniques:

- Manual and automated web application testing.
- Authentication and authorization testing.
- Configuration, TLS/SSL, and header security checks.

Client Responsibilities:

- Ensure environment stability during testing.
- Confirm the systems tested are authorized for assessment.

Limitations / Constraints:

- Production environment: testing kept non-disruptive.
- No source code access (unless explicitly provided).
- No social engineering or phishing attempts.
- Time-bound scope and testing constraints strictly followed.

Methodology:

The security assessment followed a structured, industry-standard penetration testing methodology combining **OWASP Testing Guide**, **OWASP Top 10**, and **PTES (Penetration Testing Execution Standard)**. The following phases were performed:

Reconnaissance:

- Collected publicly available information about the target.
- Identified API endpoints, subdomains, technologies, and server configurations.
- Checked for banner disclosure, outdated software, and exposed files.

Enumeration:

- Mapped all accessible directories, parameters, and API routes.
- Identified user-controlled inputs, customer identifiers, authentication endpoints, and parameter behaviors.
- Detected sensitive information leaks such as cookies, tokens, and headers.

Vulnerability Analysis:

- Tested each endpoint against OWASP Top 10 categories:
 - Authentication weaknesses
 - Authorization/IDOR
 - Input validation (XSS, injection)
 - Security misconfigurations
 - Logic flaws
 - Session & cookie issues
 - Rate limiting & brute-force weaknesses
- Performed manual analysis and tool-assisted scanning.

Exploitation:

- Executed controlled exploitation to validate real impact.

- Demonstrated IDOR data exposure, MFA manipulation, price modification, and other confirmed vulnerabilities.
- Captured PoC data and screenshots for each finding.

Post-Exploitation Verification:

- Verified the extent of access gained without causing system damage.
- Checked whether vulnerabilities could be chained for greater impact.

Environment Details:

Target URLs:

- **Main Website:** <https://www.sheba.xyz>
- **API Endpoints Tested:**
 - https://cart-api.sheba.xyz/api/v1/carts/?customer_id=
 - <https://www.sheba.xyz/profile/credit/>
 - <https://www.sheba.xyz/cart>
 - <https://www.sheba.xyz/profile/credit/>
 - Authentication-related OTP/MFA endpoints
 - Additional discovered subdomains and public services identified during reconnaissance

API Endpoints

- Customer cart retrieval API
- OTP / MFA submission endpoints
- Login, signup, and password reset endpoints
- Supporting internal service URLs identified during scanning
- Any accessible non-sensitive files (robots.txt, readme, changelogs, etc.)

Technology Stack:

Based on fingerprinting and response headers:

- **Frontend**
 - **Nuxt.js** – Server-side rendering framework built on Vue.js
 - **Vue.js** – Primary JavaScript framework
 - **Tailwind CSS** – Utility-first CSS framework
 - **Font Awesome** – Icon library
 - **Google Font API** – Web font delivery
 - **Glide.js** – Carousel/slider library
 - **SweetAlert2** – Popup/alert interface library
 - **core-js** – Polyfills for JavaScript compatibility
- **Backend / Server-Side & Infrastructure**
 - **Amazon Web Services (AWS)** – Hosting / cloud infrastructure
 - **HTTP/2** – Transport protocol detected
 - **Webpack** – Module bundler used in application build process
- **CDN & Caching**
 - **Cloudflare** – CDN and security edge services
 - **jsDelivr** – CDN for static assets

- **cdnjs** – CDN for library distribution
- **Search Technology**
 - **Algolia** – Hosted search engine service
- **Analytics & Tracking**
 - **Google Analytics** – Web traffic & behavior analytics
 - **Facebook Pixel** – Marketing/tracking pixel
 - **Amplitude** – Product analytics
 - **Sentry** – Application error monitoring
 - **Google Tag Manager** – Script/tag management
- **Ecommerce**
 - **Cart Functionality** (custom implementation detected)
- **Communication**
 - **REVE Chat** – Live chat support system
- **Web Standards & Miscellaneous**
 - **PWA (Progressive Web App)** – Mobile and performance optimization
 - **Open Graph** – Social media metadata for rich sharing

Testing Environment:

- Operating System: Kali Linux
- Browser: Firefox Developer Edition / Chrome
- Tools Used:
 - Burp Suite Community/Pro
 - Nmap
 - FFUF / Dirsearch
 - cURL
 - Subfinder / Assetfinder
 - Nuclei
 - Wappalyzer
 - Developer tools (Firefox Inspector)
 - Custom scripts and manual testing methods

Steps of Finding Bugs:

- **Initial Reconnaissance**
 - Performed subdomain enumeration, directory scanning, and technology fingerprinting.
 - Identified publicly exposed APIs, authentication endpoints, and cart-related parameters.
- **Parameter & Input Enumeration**
 - Inspected all user-controlled inputs, cookies, headers, query parameters, and form fields.
 - Observed customer_id, OTP submission parameters, and price fields accepting unsafe values.
- **Authentication & MFA Testing**
 - Observed OTP timing manipulation and ability to redirect OTP delivery to another number.
 - Tested rate limiting, OTP expiry handling, and multi-step authentication logic.
- **Authorization Testing (IDOR Checks)**

- Incremented customer_id parameter to verify access control.
 - Confirmed exposure of other users' personal details and cart information.
- **Business Logic Testing**
 - Manipulated price and quantity fields.
 - Successfully added negative values and “0” pricing to the cart.
- **Input Validation Testing**
 - Tested text fields with HTML/JS payloads to confirm self-XSS.
 - Verified injection points using browser DevTools and manual payloads.
- **Email Security Testing**
 - Checked SPF, DKIM, and DMARC configurations.
 - Identified possibilities for email spoofing.
- **Security Header & Cookie Analysis**
 - Reviewed HTTP response headers for missing security protections.
 - Detected lack of Secure and HttpOnly flags.
- **Rate Limiting & Brute Force Testing**
 - Repeated OTP attempts to confirm no lockout or rate limit enforcement.
 - Verified extended response times could be manipulated.
- **Configuration & TLS Testing**
 - Identified mixed-content issues, outdated components, and weak security headers.
 - Analyzed TLS/SSL configuration for misconfigurations.
- **Fingerprinting & Disclosure Testing**
 - Collected server, framework, and software version information via headers and banners.

Result Overview:

1. MFA OTP Redirect:

- **Expected Result:**

OTP should be sent only to the user's registered phone number.
- **Actual Result:**

Attacker can modify the request and change the phone number, causing OTP to be delivered to another number.

2. OTP Validation Bypass:

- **Expected Result:**

OTP validation must be strict, time-based, and protected from manipulation.
- **Actual Result:**

Attacker can delay the request time and interfere with the OTP validation process, weakening MFA security.

3. IDOR – Customer Data Exposure:

- **Expected Result:**

A user should only access their own cart and personal information.
- **Actual Result:**

Changing customer_id reveals other users' phone numbers, addresses, cart items, and sensitive details.

4. Price Manipulation / Business Logic Flaw:

- **Expected Result:**
Prices and quantities should be validated on the server side and not modifiable by the client.
- **Actual Result:**
Attacker can change item price or quantity (even negative values) by tampering with client-side requests.

5. Email Spoofing:

- **Expected Result:**
Email authentication (SPF/DKIM/DMARC) should prevent unauthorized email spoofing.
- **Actual Result:**
Weak or missing configurations allow attackers to send emails pretending to be from the official domain.

6. Self-XSS:

- **Expected Result:**
User inputs must be sanitized and encoded before rendering on the page.
- **Actual Result:**
Injected script runs in the browser when the user views the manipulated field.

7. Clickjacking:

- **Expected Result:**
Site should block iframe loading using X-Frame-Options or CSP.
- **Actual Result:**
Application loads inside an attacker's iframe, enabling UI redressing attacks.

8. Insecure Cookies:

- **Expected Result:**
Session cookies should use Secure, HttpOnly, and SameSite attributes.
- **Actual Result:**
Cookies are missing security flags, allowing JavaScript access and insecure transmission.

9. Text Injection:

- **Expected Result:**
All text-based fields should validate and sanitize user input.
- **Actual Result:**
Attacker can inject arbitrary text, causing UI/content manipulation.

10. Missing Security Headers:

- **Expected Result:**
Security headers (CSP, HSTS, X-Frame-Options, etc.) should be properly configured.
- **Actual Result:**
Application lacks multiple essential headers, increasing attack surface.

11. TLS/SSL Issues:

- **Expected Result:**
TLS configuration should enforce strong protocols, ciphers, and HSTS.
- **Actual Result:**
Weak or outdated TLS settings make the system vulnerable to MITM and downgrade attacks.

12. Fingerprinting & Information Disclosure:

- **Expected Result:**
The server should **hide internal technology details** such as web server version, framework version, or OS information.
- **Actual Result:**
The response reveals **server fingerprints/banner information** (e.g., server type, framework, or version), which allows attackers to profile technologies and target known vulnerabilities

Findings Summary Table

Vulnerability	Severity	Justification
MFA OTP Redirect	Medium	OTP delivery can be redirected to an attacker-controlled number, increasing risk of unauthorized access.
OTP Validation Bypass	Critical	Allows login without completing OTP verification, enabling full account takeover.
IDOR – Customer Data Exposure	Critical	Unauthorized access to other customers' personal and financial information, violating privacy and compliance requirements.
Business Logic Flaw – Price & Quantity Manipulation	Critical	Enables manipulation of item prices and negative quantities, directly causing financial loss and fraudulent transactions.
Email Spoofing (SPF/DKIM/DMARC Weakness)	High	Attackers can impersonate the brand to deliver phishing or social-engineering attacks.
Self-XSS	Informational	Exploitable only by tricking the user into executing code; limited security impact.
Clickjacking	Medium	Lack of frame protection allows forced user actions (e.g., purchases, profile changes).
Insecure Cookies (Missing Secure & HttpOnly Flags)	Medium	Increases risk of session hijacking via network sniffing or script access.
Text Injection / Content Manipulation	Low	Only affects UI text; no script execution or significant security impact.
Missing Security Headers	High	Absence of CSP, X-Frame-Options, HSTS, etc., increases risk of XSS, clickjacking, and protocol downgrade attacks.
TLS/SSL Misconfigurations	High	Weak or outdated cryptographic configurations may allow MITM attacks and data interception.
Fingerprinting / Banner Disclosure	Medium	Exposes server technologies and versions, aiding attackers in reconnaissance and targeted exploitation.

Detailed Vulnerability Reports

1. MFA Issues – OTP Redirect

Severity: Medium

Description:

The application uses a One-Time Password (OTP) sent via SMS as a factor for authentication (MFA). The flaw lies in the **server's failure to validate the user's authorization** to change the phone number associated with the current session during the OTP request.

Reproduction:

- Intercept Request: Capture the API request to the OTP generation endpoint (/api/shoot-otp) using a web proxy tool.
- Identify Parameter: Locate the mobile parameter within the request body, which specifies the destination for the OTP.
- Tamper Value: Substitute the victim's legitimate mobile number with an attacker-controlled phone number.
 - Original Body Example: {"mobile": "+8801581483220", ...}
 - Modified Body Example: {"mobile": "+8801746777198", ...}
- Forward Request: Send the modified request to the server.
- Verification: Confirm that the server returns a successful response (e.g., HTTP 200 OK) and the OTP code is delivered to the attacker's device, successfully bypassing the MFA protection for the victim's account.

Proof of Concept:

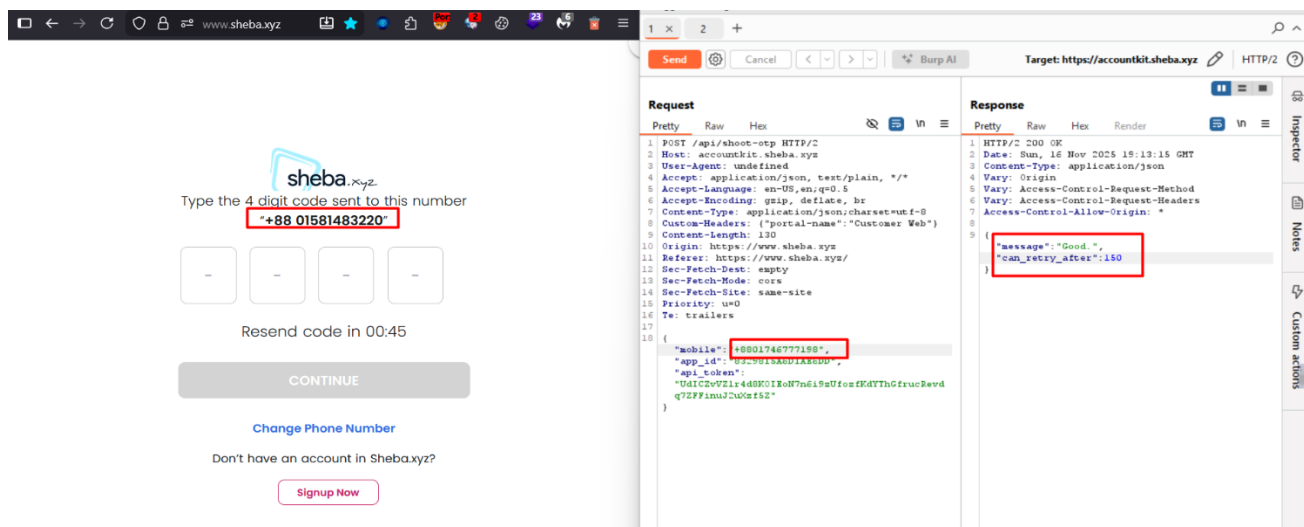


Figure 1.1: OTP Redirect

Impact:

- **Critical Severity:** This vulnerability completely defeats the purpose of Multi-Factor Authentication.

- **Account Takeover (ATO):** An attacker gains the one piece of information (the OTP) needed to complete the authentication process for the victim's account. This allows the attacker to log in as the victim and gain full access to their data, settings, and potentially perform unauthorized actions.
- **No Victim Interaction Required:** Unlike SIM Swapping or social engineering attacks, this exploit requires **no interaction** from the victim; the attacker simply redirects the code through a single malicious API call.

Mitigation:

- Server-side validation of phone number.
- Strict OTP binding to session + rate limiting.
- Block parameter tampering.

2. OTP Validation Bypass

Severity: High/Critical

Description:

The OTP verification mechanism relies on client-provided parameters (`can_retry_after`: 180) to enforce time-based expiration. Attackers can intercept and modify this value to extend the OTP validity period beyond the intended security limits.

Reproduction Steps

1. **Request & Intercept:** Initiate the OTP request and intercept the server's HTTP response (e.g., from `POST /api/shoot-otp`).
2. **Tamper Parameter:** Locate and modify the client-facing time enforcement parameter (`"can_retry_after"`).
 - **Original:** `"can_retry_after": 150`
 - **Modified:** Change value to an extended duration, e.g., `"can_retry_after": 900`
3. **Forward Response:** Allow the modified response to reach the client.
4. **Verification:** Wait for the original expiration time (150 seconds) to pass. Submit the valid OTP. The successful validation confirms that the server is honoring the client-manipulated expiration time, effectively extending the OTP's lifespan and bypassing the intended security measure.

Proof of Concept:

During normal operation, when an OTP is requested for the number +88 01581483220, the server returns a response containing `"can_retry_after": 150`. This value represents the intended 150-second (2.5-minute) waiting period before requesting another OTP, and the OTP itself expires within this time frame.

By intercepting the server's response using Burp Suite, the attacker can alter this parameter before it reaches the client. Changing the value from 150 to a higher number, such as 900, forces the system to treat the OTP as valid for a much longer time than intended. After modifying the value, the attacker forwards the manipulated response, successfully altering the OTP timing behavior.

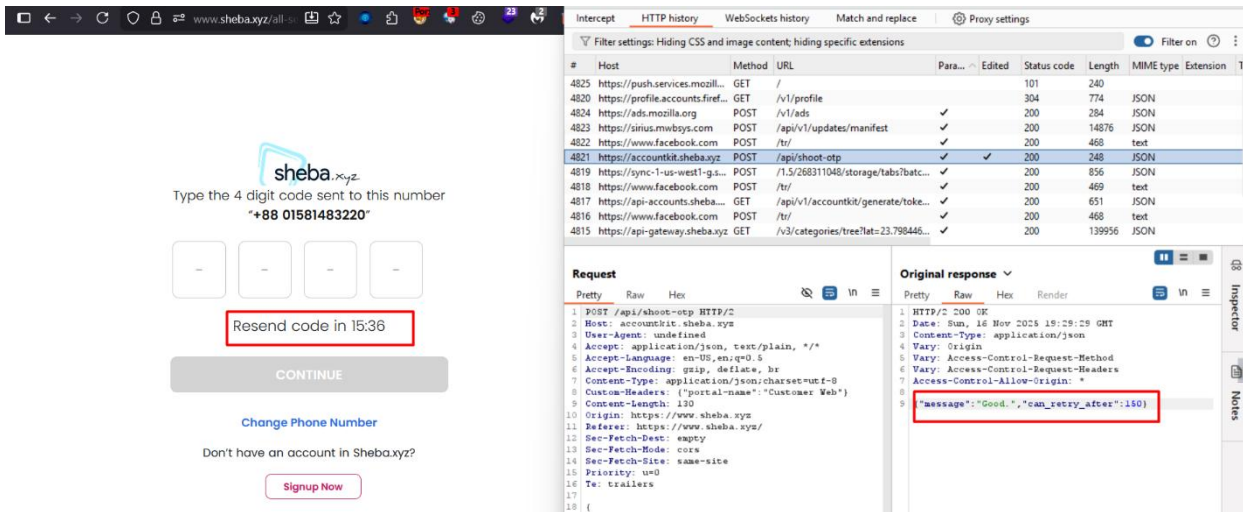


Figure 2.1: OTP Validation Bypass

Impact:

- Authentication Bypass: Attackers can use expired OTPs
- Reduced Security: Time-based OTP protection is defeated
- Account Takeover: Potential unauthorized access to user accounts
- Business Logic Compromise: Core security controls can be manipulated

Mitigations:

- Remove client-controlled timing parameters - Eliminate `can_retry_after` from API responses
 - Implement server-side expiration tracking - Store OTP creation time in database
 - Validate against server timestamp - Compare current time with stored OTP creation time
 - Store OTP created_at and expires_at timestamps server-side
 - Use server time for all expiration calculations
 - Add rate limiting on OTP verification attempts
 - Log all OTP validation attempts for monitoring
 - Implement cryptographic signatures for time-sensitive data
- Conduct security code reviews for authentication flows

3. IDOR – Customer Details Exposure

Severity: Critical

Description:

An **Insecure Direct Object Reference (IDOR)** vulnerability was identified in multiple API endpoints that use the `customer_id` parameter to retrieve user-specific data. The application fails to enforce proper authorization checks, allowing any authenticated user—or in some cases, even an attacker with minimal access—to manipulate the `customer_id` value and retrieve the personal information of other customers.

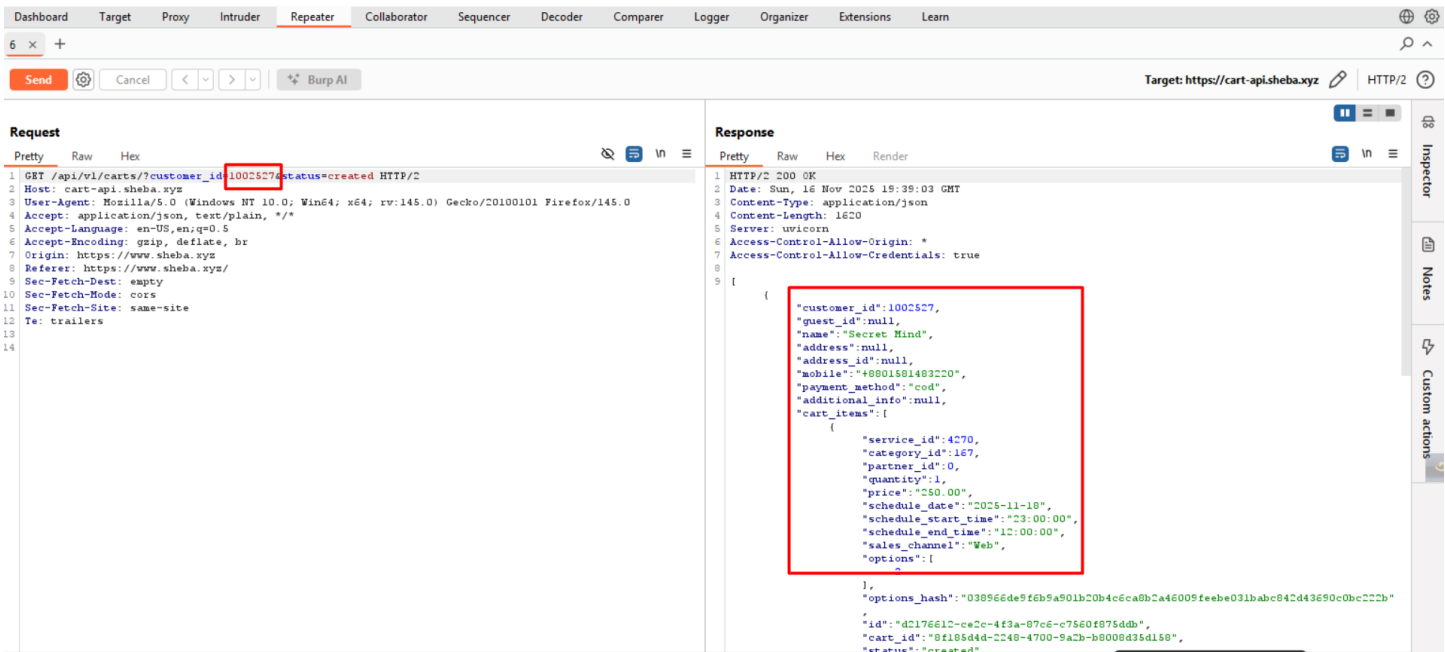
By modifying the `customer_id` parameter to another valid ID, the server returns a successful **HTTP 200 OK** response along with sensitive details belonging to that user. Exposed information includes full name, phone number, address, cart items, order history, and other personally identifiable data.

This vulnerability represents a **direct breach of confidentiality** and puts thousands of customers at risk. An attacker can easily enumerate customer IDs, automate data harvesting, and build complete profiles of users without detection.

Reproduction:

1. **Identify Endpoint:** Access an authenticated endpoint that retrieves customer-specific data, such as: `/v2/customers/<customer_id>/...` or `/v1/carts/<customer_id>/...`
2. **Capture Request:** Intercept the API request using a proxy tool. Note the integer parameter corresponding to the user ID (e.g., `customer_id=1002527` in the image).
3. **Parameter Tampering:** Modify the legitimate `customer_id` with an arbitrary, sequential, or guessed numerical ID belonging to a different user.
 - **Original ID:** 1002527
 - **Tampered ID:** 1002546 (or any valid, external ID)
4. **Verification:** Send the tampered request. The server returns an HTTP 200 OK response containing the full personal and sensitive details (e.g., name, mobile, address, `cart_items`) associated with the tampered `customer_id`.

Proof of Concept:



The screenshot displays the Burp Suite interface with the 'Repeater' tab selected. The target URL is `https://cart-api.sheba.xyz`. The request is an HTTP 2.0 GET to `/api/v1/carts/?customer_id=1002527&status=created`. The response is an HTTP 2.0 OK with a JSON body containing customer and cart details. A red box highlights the JSON response, showing fields like `customer_id`, `name`, `mobile`, and `cart_items`.

```
Request
Pretty Raw Hex
1 GET /api/v1/carts/?customer_id=1002527&status=created HTTP/2
2 Host: cart-api.sheba.xyz
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:145.0) Gecko/20100101 Firefox/145.0
4 Accept: application/json, text/plain, */*
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Origin: https://www.sheba.xyz
8 Referer: https://www.sheba.xyz/
9 Sec-Fetch-Dest: empty
10 Sec-Fetch-Mode: cors
11 Sec-Fetch-Site: same-site
12 Te: trailers
13
14

Response
Pretty Raw Hex Render
1 HTTP/2 200 OK
2 Date: Sun, 16 Nov 2025 19:39:03 GMT
3 Content-Type: application/json
4 Content-Length: 1620
5 Server: uvicorn
6 Access-Control-Allow-Origin: *
7 Access-Control-Allow-Credentials: true
8
9 {
10   {
11     "customer_id": 1002527,
12     "guest_id": null,
13     "name": "Secret Mind",
14     "address": null,
15     "address_id": null,
16     "mobile": "+9801501483220",
17     "payment_method": "cod",
18     "additional_info": null,
19     "cart_items": [
20       {
21         "service_id": 4270,
22         "category_id": 167,
23         "partner_id": 0,
24         "quantity": 1,
25         "price": "250.00",
26         "schedule_date": "2025-11-18",
27         "schedule_start_time": "23:00:00",
28         "schedule_end_time": "12:00:00",
29         "sales_channel": "Web",
30         "options": {
31           "options_hash": "038966de9f6b5a901b20b4c6ca0b2a46009feebe031babc842d43690c0bc222b",
32           "id": "d2176612-ca2c-4f3a-87c6-c7560f875ddb",
33           "cart_id": "98105d4d-2248-4700-9a2b-b8009d35d150",
34           "created": true
35         }
36       }
37     ]
38   }
39 }
```

Figure 3.1: Attacker customer id

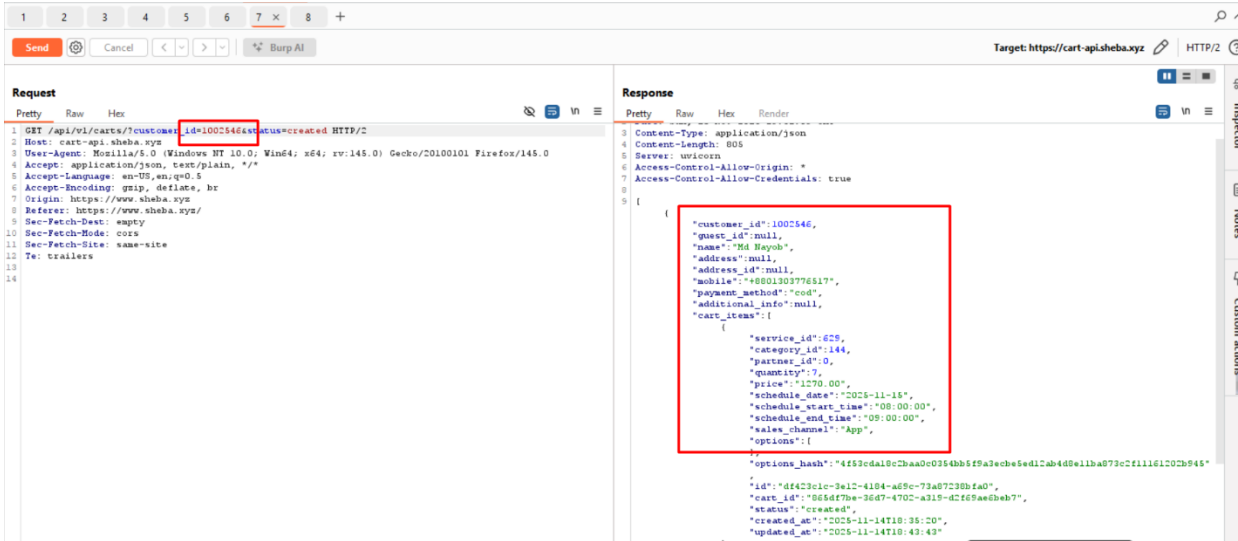


Figure 3.2: Another customer id

Impact:

- **Data Breach:** Unauthorized access and exposure of extensive customer records.
- **PII Leakage:** Exposure of sensitive Personally Identifiable Information (Names, Addresses, Phone Numbers, Emails).
- **Confidentiality Violation:** Breach of user data confidentiality and privacy.
- **Regulatory Fines:** Risk of massive penalties under laws like GDPR and CCPA.
- **Legal Action:** Potential for class-action lawsuits from affected customers.
- **Identity Theft Risk:** Exposed data enables attackers to commit identity fraud against victims.
- **Phishing/Social Engineering:** Data used to craft targeted, highly effective phishing campaigns.
- **Reputational Damage:** Severe loss of public trust and brand credibility.
- **Customer Churn:** Users may leave the service due to security concerns.
- **Forensic Costs:** High expense for incident response and mandated security audits.

Mitigation:

- **Strict Server-Side Authorization Check (Must-Do)**
 - **Verify Ownership:** Before retrieving any customer data, the server must always check that the authenticated user's ID matches the requested customer ID.
 - **Retrieve from Session:** Do not trust the ID supplied in the request; use the user ID securely stored in the active session for the database lookup.
- **Use Indirect Object References**
 - **Obfuscate IDs:** Replace sequential database IDs in URLs and API parameters with non-guessable identifiers like UUIDs (Globally Unique IDs).
 - **Secure Mapping:** Maintain a secure, server-side-only layer to map the public UUIDs back to the internal database IDs.
- **Implement Robust Access Control**
 - **Role-Based Access:** Enforce Role-Based Access Control (RBAC) to ensure users can only access endpoints and data permitted by their assigned role (e.g., standard user, administrator).

- Default Deny: Adopt a security principle where all access is denied unless explicitly granted.

4. Business Logic Flaw – Price Manipulation

Severity: High

Description

The cart update API endpoint exhibits a critical **Business Logic Flaw** due to a failure in server-side input validation for core commerce parameters. Specifically, the system allows authenticated users to submit **non-positive (zero or negative) values** for product **quantity** or **price**.

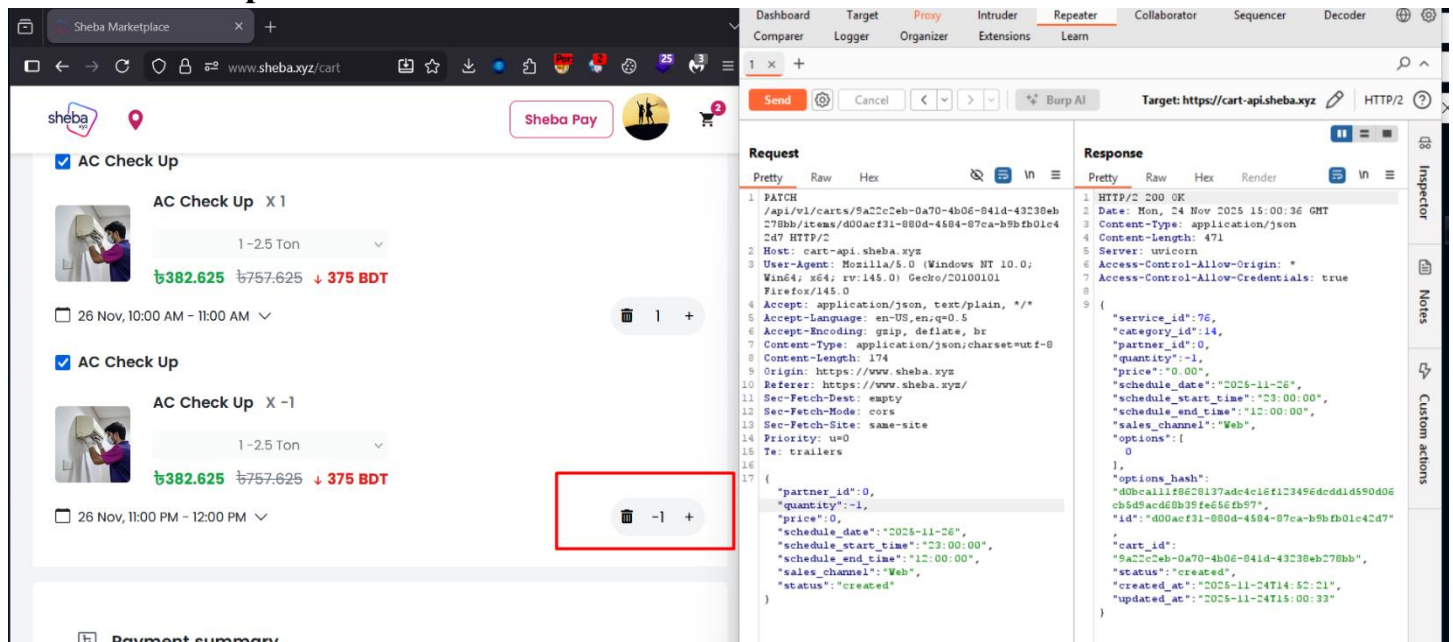
This flaw permits an attacker to:

- Set the quantity to **zero (0)**, potentially allowing the transaction to proceed with a total cost of zero, resulting in a **free purchase**.
- Set the quantity or price to a **negative integer (e.g., -1)**. If the backend uses this value in the final calculation, it can lead to a **price reversal**, resulting in the site processing a negative charge and effectively crediting the attacker's account.

Reproduction:

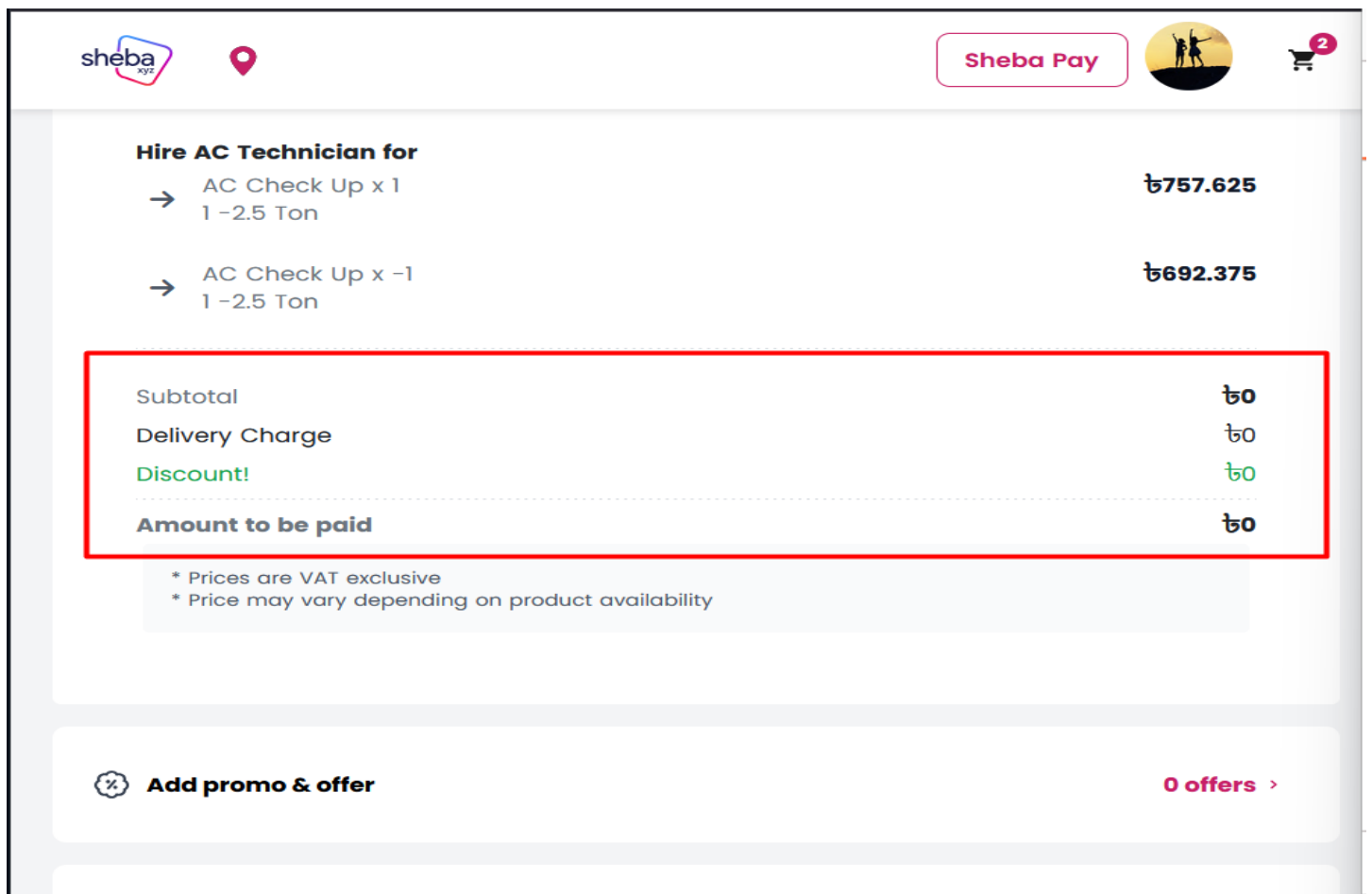
- **Add Item to Cart:** Add a purchasable service or item to the online shopping cart.
- **Intercept Request:** Use an HTTP proxy tool (e.g., Burp Suite) to intercept the request that modifies the cart contents (e.g., clicking the 'Remove' or 'Quantity Down' button shown in the image). The endpoint is typically a PATCH or PUT request to a cart ID URL.
- **Identify and Tamper Parameter:** Locate the parameter controlling the item quantity ("quantity"). Modify this value to an unacceptable input:
 - **Original Body Example (Normal reduction):** {"quantity": 0, ...}
 - **Tampered Body Example (Exploitation):** {"quantity": -1, ...} or {"quantity": 0, ...}
- **Forward Request:** Send the request with the manipulated quantity (e.g., -1) to the server.
- **Verification:** The server returns a successful response (HTTP 200 OK) and processes the transaction or cart update based on the invalid quantity. This demonstrates that the backend failed to enforce **positive, non-zero validation** on the critical quantity parameter, potentially allowing:
 - **Zero Quantity:** Processing a transaction for free.
 - **Negative Quantity:** Receiving a credit or reducing the total cost by the item's price multiplier.

Proof of Concept:



The screenshot shows the Sheba Marketplace interface with two 'AC Check Up' items. Each item has a quantity of 1 and a price of ₹382.625. A red box highlights the quantity input field, which allows entering '-1'. To the right, a Burp Suite request and response are shown. The request is a PATCH to the API endpoint /api/v1/carts/9a22c2eb-0a70-4b06-841d-43238eb278bb/items/d00acf31-880d-4584-87ea-b9fb01c42d7. The response is a 200 OK with a JSON body containing details about the service, including a partner_id, category_id, price, and schedule.

Figure 4.1: Logic flaws Negative product quality



The screenshot shows the Sheba Marketplace checkout page. The page displays a summary of the order, including the subtotal, delivery charge, discount, and the amount to be paid. A red box highlights the 'Amount to be paid' section, which shows a total of ₹0. Below the summary, there are two footnotes: '* Prices are VAT exclusive' and '* Price may vary depending on product availability'. At the bottom, there is a section for 'Add promo & offer' with a button labeled '0 offers >'.

Figure 4.2: submit a quantity of -1 for a service or product.

Impact:

- **Financial Loss / Service Theft (Critical):** The primary impact is the ability for an attacker to obtain services (like the AC Check Up) or physical goods without making any payment, leading to direct revenue loss for the company.
- **Data Integrity Compromise (High):** Processing negative quantities corrupts order data, inventory counts (if applicable), and financial records, leading to major reconciliation issues.
- **System Abuse (High):** This vulnerability can be used for widespread abuse, potentially generating large numbers of fraudulent, free orders.

Mitigation:

The mitigation for this logic flaw involves implementing strict **server-side validation** for all quantity and price-related inputs.

- **Strict Quantity Validation (Mandatory):**
 - The server must **reject any quantity** value that is zero or negative (quantity > 0). This check must occur on the server side immediately upon receiving the cart update or order placement request.
 - Only allow positive integers or specific fractional values if applicable to the product/service.
- **Server-Side Price Calculation:**
 - **Do not trust the client** (browser) to send the price or total. The server should retrieve the current price of the item from its secure database using the product ID and perform all final subtotal and total calculations itself.
- **Data Type Enforcement:**
 - Ensure that all quantity fields in API requests and database tables are designed to handle only unsigned integers where appropriate, or have explicit constraints to prevent negative values.
 - Use the tag to illustrate the necessity of server-side checks.
- **Transaction Validation:**
 - Implement a final check before processing payment to ensure the Amount to be paid is greater than a minimum threshold (e.g., greater than ₹0).

5. Email Spoofing (SPF, DKIM, DMARC Weaknesses)

Severity: Medium

Description:

The application's domain, sheba.xyz, is susceptible to **Email Spoofing**, allowing an attacker to send emails that appear to originate from a legitimate address within the domain (e.g., admin@sheba.xyz).

The evidence provided confirms this by demonstrating that the SMTP server accepts an arbitrary address in the **MAIL FROM** field, which is used to specify the sender's envelope address.

Reproduce:

- **MX Record Enumeration**
 - Checked domain's mail server configuration.
Command: (dig +short MX sheba.xyz)
 - Identified active mail servers for handling inbound email.

- **SPF Record Check**
 - Retrieved SPF record to analyze allowed senders.
Command: (dig +short TXT sheba.xyz)
 - SPF ended with ~all (SoftFail), meaning spoofing is possible.
- **DMARC Verification**
 - Checked if DMARC policy exists or is enforced.
Command: (dig +short TXT _dmarc.sheba.xyz)
 - Found **no enforcement (p=none / or missing)** → domain vulnerable.
- **DKIM Verification**
 - Checked for DKIM selectors.
 - **Common selector check:** (dig +short TXT default._domainkey.sheba.xyz)
 - **No valid DKIM key found** → signing not enforced.
- **Safe Email Spoofing Proof-of-Concept (PoC)**
 - Used **Mailtrap** for safe testing (no real users targeted).
 - Collected SMTP credentials from Mailtrap dashboard.
- **Send Spoofed Email**
 - Sent a forged email using the domain sheba.xyz as sender.
Command used:

```
swaks --from admin@sheba.xyz \  
--to test@mailtrap.io \  
--server smtp.mailtrap.io \  
--port 2525 \  
--auth \  
--auth-user YOUR_USER \  
--auth-password YOUR_PASS
```
- **Result Verification**

Checked Mailtrap inbox:

 - Email successfully delivered **using fake sender address**.
 - No SPF, DKIM, or DMARC rejection.
 - Confirmed **sheba.xyz is vulnerable to spoofing**.

Proof of Concept:

```
(Error@secretmind)-[~/Tools/r3con]  
$ dig +short TXT sheba.xyz  
"google-site-verification=3UAQIP4sEyZE3Ad2I-4cmuMwKAXt0_hG2B7pQW_m6w"  
"v=spf1 include:sendclean.net include:_spf.us.sendclean.net include:mailgun.org include:_spf.google.com include:_spf.mlsend.com ~all"  
"google-site-verification=t92t05sqXhzt-gn0qajgmMQ6itACP_03zCHLuqxpx4g"  
"google-site-verification=RjKMg7IJi0eo7vgA_n0-0THEI_KRzVC0UiPare0VXqM"  
(Error@secretmind)-[~/Tools/r3con]
```

Figure 5.1: SPF invalid — lookup limit exceeded

```
(Error@secretmind)-[~/Tools/r3con]
$ swaks --from admin@sheba.xyz \
  --to test@mailtrap.io \
  --server smtp.mailtrap.io \
  --port 2525 \
  --auth \
  --auth-user 330c5a16184883 \
  --auth-password 3d230ab2a32ea7
Trying smtp.mailtrap.io:2525 ...
Connected to smtp.mailtrap.io.
220 smtn.mailtrap.io ESMTP ready
→ EHLO secretmind
← 250-smtp.mailtrap.io
← 250-SIZE 5242880
← 250-PIPELINING
← 250-ENHANCEDSTATUSCODES
← 250-8BITMIME
← 250-DSN
← 250-AUTH PLAIN LOGIN CRAM-MD5
← 250 STARTTLS
→ AUTH CRAM-MD5
← 334 PDM0Nzc2NTgyNi4xNzYzNTg3Mzk4QHNTdHAubWFpbHRyYXAuaW8+
→ MzMwYzVhMTYxODQ4ODMgM2IyZTZiMjJlYjYyZjYyYWE5YzY3NTg3NDI1YjY=
← 235 2.0.0 OK
→ MAIL FROM:<admin@sheba.xyz>
← 250 2.1.0 Ok
→ RCPT TO:<test@mailtrap.io>
← 250 2.1.0 Ok
→ DATA
← 354 Go ahead
→ Date: Thu, 20 Nov 2025 03:15:49 +0600
→ To: test@mailtrap.io
→ From: admin@sheba.xyz
→ Subject: test Thu, 20 Nov 2025 03:15:49 +0600
→ Message-Id: <20251120031549.033650@secretmind>
→ X-Mailer: swaks v20240103.0 jetmore.org/john/code/swaks/
→ This is a test mailing
→ .
← 250 2.0.0 Ok: queued
→ QUIT
← 221 2.0.0 Bye
Connection closed with remote host.
```

Figure 5.2: spoofed email successfully sent

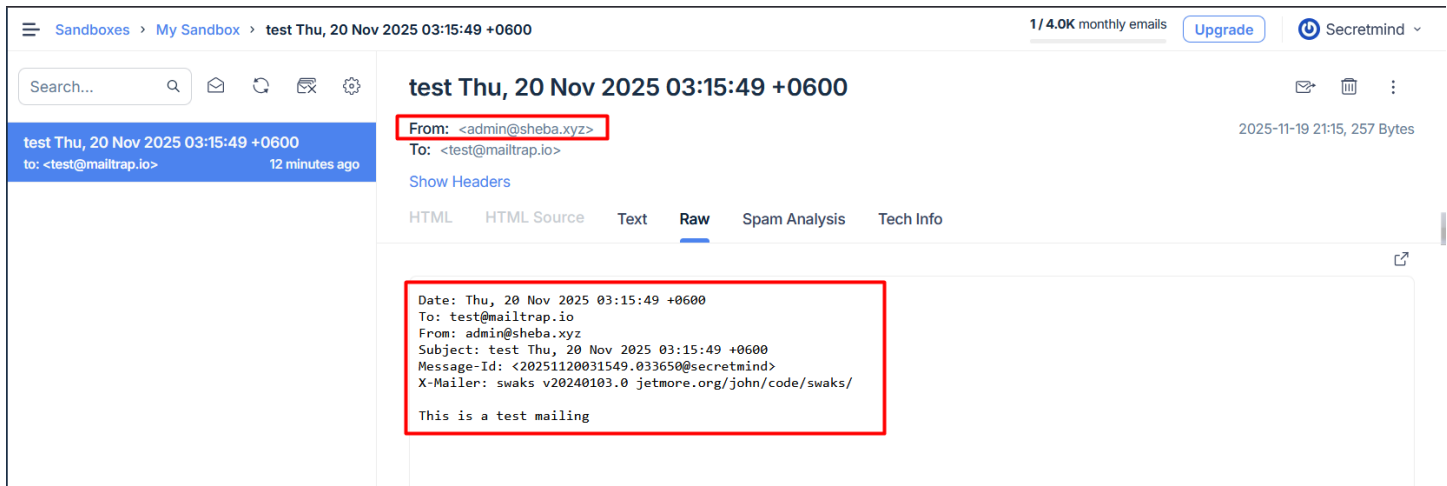


Figure 5.3: SPF Lookup Limit Bypass

Impact:

The impact primarily concerns **reputation, trust, and security**.

- **Brand Impersonation:** Attackers can send emails that look legitimate, damaging your brand's credibility.
- **Phishing/BEC:** Increased risk of successful **phishing** and **Business Email Compromise (BEC)** attacks against employees and customers.
- **Credential Theft:** Fraudulent emails are used to trick users into giving up login credentials.
- **Malware Delivery:** The spoofed emails can deliver ransomware or other malicious software.
- **Loss of Trust:** Customers and partners lose confidence in your official communication channels.
- **Poor Deliverability:** Your legitimate emails may be flagged as **spam** by recipient servers (e.g., Gmail, Outlook), severely impacting business communication.
- **Financial Loss:** Direct monetary losses resulting from fraudulent transfers or successful BEC scams.

Mitigation:

The mitigation requires configuring three complementary security records in DNS.

1. Implement/Harden SPF (Sender Policy Framework)

- **Define Authorized Senders:** Ensure the DNS record lists **all** servers and third-party services authorized to send mail for your domain.
- **Enforce Strict Policy:** Change the final mechanism from a weak policy like ~all (SoftFail) or ?all (Neutral) to **-all (Fail)** to instruct recipients to reject unauthorized emails.

2. Implement and Use DKIM (DomainKeys Identified Mail)

- **Sign Outgoing Mail:** Configure your mail servers and services to digitally **sign all outgoing emails** with a private key.
- **Publish Public Key:** Publish the corresponding public key as a DNS TXT record for verification.

3. Implement and Enforce DMARC (Domain-based Message Authentication, Reporting, and Conformance)

- **Publish DMARC Record:** Create a DMARC record in your DNS (e.g., _dmarc.yourdomain.com).
- **Align Checks:** Ensure both SPF and DKIM checks are configured to align with the "From" domain.
- **Set Strong Policy:** Gradually move the policy from monitoring (p=none) to quarantining (p=quarantine), and finally to **p=reject** to block all unauthenticated emails.

6. Self-XSS

Severity: Medium

Description:

During a security assessment of the sheba.xyz platform, a JavaScript alert execution was observed when manually running JavaScript code through the browser's URL bar or developer console.

Reproduce:

- Open browser dev tools (F12 → Console).
- Manually type: javascript:alert(100)
- Press Enter.
- An alert popup appears displaying 100.

Proof of Concept:

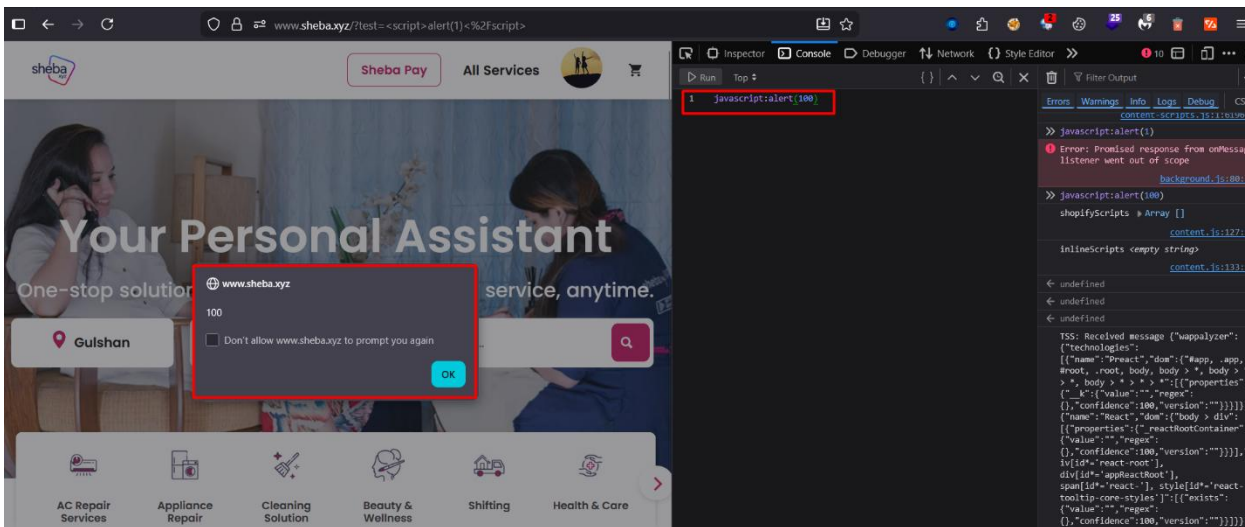


Figure 6.1: Script executes.

Impact:

Self-XSS has no impact and is not exploitable.

It requires the victim to intentionally run arbitrary JavaScript in their own browser, which is not a valid attack vector.

Modern browsers warn users against this behavior (e.g., Chrome's "Stop! This is a self-XSS attack" message), confirming that this is **not a web application vulnerability**.

Mitigation:

- Implement proper output encoding (HTML-encode user input before displaying it).
- Apply strict input validation and sanitize all user-controlled fields.
- Disable rendering of HTML tags inside text fields.
- Use Content Security Policy (CSP) to reduce script execution risks.
- Add warnings against entering scripts in input fields (optional but recommended).

7. Clickjacking

Severity: Medium

Description:

Clickjacking is a User Interface (UI) redress attack where an attacker tricks a user into clicking on an element of a hidden page, effectively executing an action they did not intend.

The vulnerability exists because the target website, **sheba.xyz**, does not send appropriate security headers to **prevent its content from being loaded inside a frame (<iframe>)** on an external domain.

The uploaded image confirms this: the target site's content (the "My Cart" and "Payment summary" pages) is successfully loaded inside an **external test frame**. An attacker could place this frame transparently over a malicious website, coercing the user into clicking a sensitive button (e.g., "Confirm Order" or "Change" their account details) on the hidden, framed page.

Reproduce

- Create a file named clickjack-poc.html on your machine.
- Paste the following PoC code into the file.
- Replace : `https://www.sheba.xyz/cart/SENSITIVE-ACTION` with the endpoint/page you found vulnerable.
- Open the file in a browser.
- If the target page loads inside the iframe and the button becomes clickable → Clickjacking is confirmed.

Proof of Concept:



Clickjacking Test Frame

If the target website loads inside this frame, it is VULNERABLE.

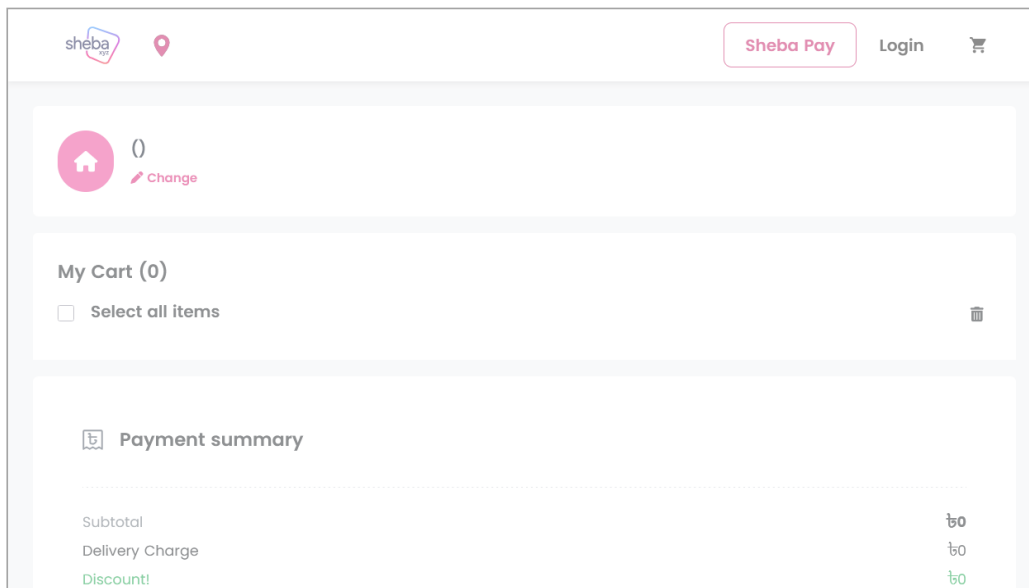


Figure 7.1: Clickjacking

Impact:

The impact depends on the sensitive actions a user can perform on the framed page.

- **Unauthorized Financial Transactions:** Forcing clicks on "Buy Now" or "Confirm Order" buttons, leading to unauthorized purchases.
- **Account Settings Modification:** Tricking users into clicks that change the account's primary email, password, or phone number, enabling account takeover.
- **Data Leakage:** Forcing clicks on invisible buttons that submit forms containing sensitive profile information.
- **System Integrity Compromise:** Forcing users to change critical security settings or disable controls.
- **Reputational Damage:** The use of the service for unauthorized actions harms user trust.

Mitigation:

The mitigation requires implementing specific HTTP response headers to instruct the browser to deny framing from external domains.

- **Implement X-Frame-Options (XFO):** Set this header on all pages to restrict framing.
 - **X-Frame-Options: DENY: (Recommended)** Prevents the page from being displayed in any frame, regardless of the site attempting to frame it.
- **Implement Content-Security-Policy: frame-ancestors (CSP):** This is the modern, more robust solution.
 - **Content-Security-Policy: frame-ancestors 'none': (Recommended)** Prevents the page from being framed by any domain.
 - *Alternative:* Use Content-Security-Policy: frame-ancestors 'self' to only allow framing from the same origin.

8. Lack of Secure & HttpOnly Cookies

Severity: Medium/Low

Description:

This vulnerability is a configuration failure in how the application sets and manages cookies used for authentication and tracking. The cookies, particularly those storing sensitive session information (like the user cookie and potentially others observed in the storage tab), are missing two critical security attributes:

- **Missing HttpOnly Flag:** This flag prevents client-side scripts (like JavaScript) from accessing the cookie. Since the flag is missing, if an attacker successfully executes a **Cross-Site Scripting (XSS)** attack, their malicious script can easily read and steal the cookie's value.
- **Missing Secure Flag:** This flag tells the browser to **only** send the cookie over an encrypted **HTTPS** connection. Since the flag is missing, the browser might send the cookie over an unencrypted **HTTP** connection (if one is ever used), exposing the sensitive session data to interception by a network attacker (Man-in-the-Middle attack).

The console output showing document.cookie successfully reading the entire cookie string and the **"false"** value under the HttpOnly column in the storage tab confirm this finding.

Reproduction:

- **Analyze Cookies:** Navigate to the browser's Developer Tools (Storage or Application tab) to inspect the cookies set for the target domain (www.sheba.xyz).
- **Verify Missing Flags:** Observe that critical cookies (e.g., _ga, g_auth, user_token) are set with the value **false** under both the **HttpOnly** and **Secure** columns.

- **Confirm HttpOnly Bypass:** Confirm client-side accessibility by navigating to the browser's Console and executing the command `document.cookie`. The successful output of all cookies, including sensitive session tokens, demonstrates the immediate threat of XSS-based cookie theft.

Proof of Concept:

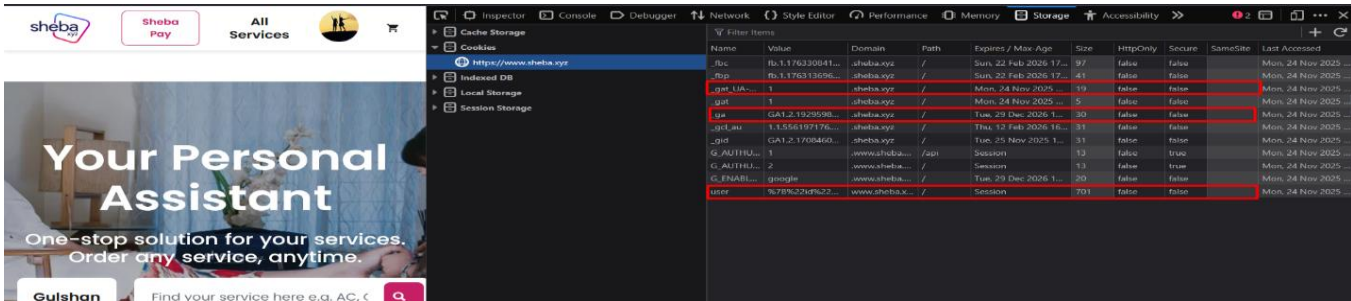


Figure 8.1: Missing Secure Flags

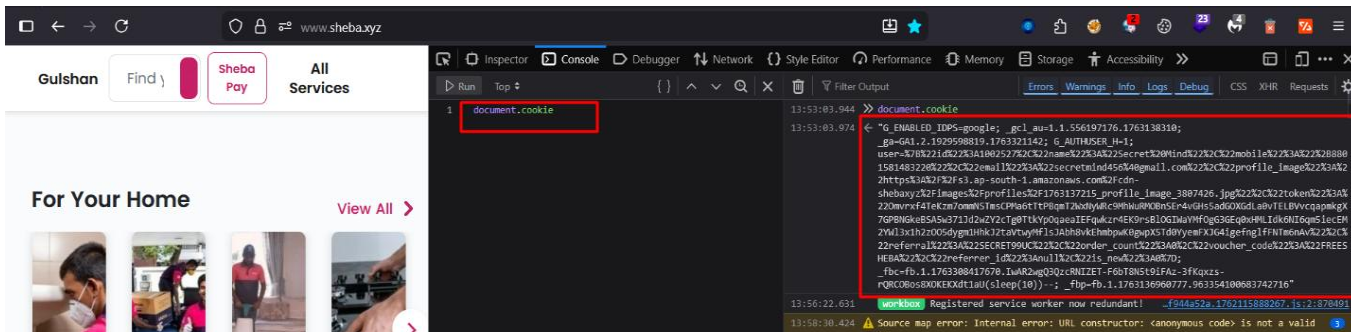


Figure 8.2: Sensitive User Data Stored in Plain Cookie

Impact:

This finding is classified as **High Severity** because it provides the primary vector for session hijacking.

- **Session Hijacking/Account Takeover:** The most severe impact. An attacker who executes a successful XSS payload can steal the session cookie, use it to impersonate the legitimate user, and bypass all authentication measures (including MFA, if already logged in).
- **Credential Theft:** The cookies often contain encoded user IDs, tokens, or other sensitive identifiers that, once stolen, can be used for further reconnaissance or direct account access.
- **Man-in-the-Middle (MITM) Exposure:** Without the Secure flag, the session cookie is vulnerable to being passively intercepted and captured by an attacker eavesdropping on the network traffic if the application ever loads over unencrypted HTTP.
- **Data Exposure:** Any sensitive information stored directly in the cookie becomes vulnerable to client-side theft.

Mitigation:

Mitigation requires changing the server-side code that issues the cookies to include the necessary flags.

- **Enforce HttpOnly Flag (Critical):** Set the `HttpOnly` attribute to true for **all** session and sensitive authentication cookies. This prevents client-side script access, blocking most XSS-based session theft.

- *Example:* Set-Cookie: sessionId=abc12345; HttpOnly;
- **Enforce Secure Flag (Mandatory):** Set the Secure attribute to true for all cookies. This ensures cookies are transmitted only when the connection is encrypted (HTTPS).
- *Example:* Set-Cookie: sessionId=abc12345; Secure;
- **Minimize Cookie Content:** Ensure that cookies contain only necessary, non-sensitive identifiers (e.g., a session ID). Avoid storing PII or any direct credential material within the cookie itself.
- **Use Strong Session Management:** Regenerate session IDs upon successful login and periodically to limit the window of opportunity for an attacker using a stolen cookie.

9. Text Injection (Content Injection)

Severity: Medium

Description:

Text Injection occurs when the application accepts un-sanitized user input and reflects it directly back into the dynamic content of the webpage. The screenshot demonstrates that the user was able to submit the string "**Text Injection**" (likely via a URL parameter, form field, or API request), and the application subsequently rendered this exact string in a section of the user interface—specifically, the "**Voucher ID**" field—without performing proper encoding or validation.

While the screenshot only shows simple text being injected, this vulnerability is often the precursor to a more dangerous flaw like **Cross-Site Scripting (XSS)**. The current finding means the application trusts and displays unverified data, which is a fundamental security mistake.

Reproduction

1. **Identify Endpoint:** Access the credit profile page using the following base URL:
`https://www.sheba.xyz/profile/credit/`
2. **Inject Payload:** Append a non-malicious test string to the URL path after `/credit/`.
 - **Example URL:** `https://www.sheba.xyz/profile/credit/VOUCHER-TEST-999`
3. **Execute:** Navigate to the modified URL.
4. **Verification:** Observe the page content. The raw, un-encoded string (VOUCHER-TEST-999 or the original **Text Injection**) will be reflected directly into the "Voucher ID" field, confirming the application's failure to sanitize data from the URL path before rendering it to the client.

Proof of Concept:

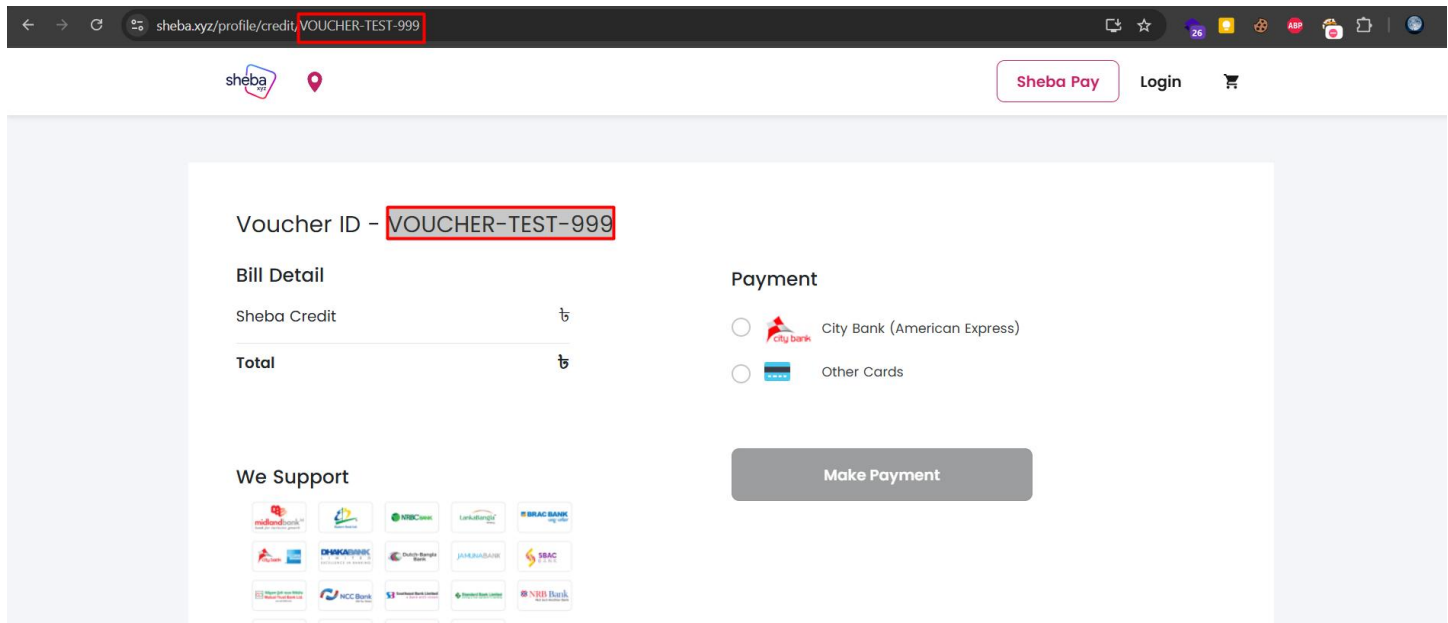


Figure 9.1: Content Injection

Impact

The impact of simple text injection is generally lower than XSS, but it still poses risks, particularly for user trust and potential confusion.

- **Defacement and User Confusion:** An attacker can inject misleading or abusive text into the page (e.g., promotional messages, misleading error codes, or offensive language). This undermines the site's authority and causes user distrust.
- **Phishing/Social Engineering:** An attacker can inject text designed to look like legitimate system messages (e.g., "Your account has been suspended, click here to re-verify") to trick users into revealing credentials or clicking malicious links.
- **Malicious Link Injection:** The attacker could inject full URLs, making them clickable if the browser automatically interprets them as links.
- **Precursor to XSS:** If the injection point allows HTML tags, the severity escalates immediately to **Cross-Site Scripting (XSS)**, which is a **Critical** finding allowing for session hijacking.

Mitigation:

Mitigation for Text Injection involves applying the principles of **Output Encoding** and **Input Validation** to ensure no raw, unverified data is rendered to the user.

- **Output Encoding (Primary Defense):** Before rendering user input to the HTML page, **always encode** the output. This converts potentially dangerous characters (like `<`, `>`, and `&`) into their harmless HTML entity equivalents (e.g., `<`, `>`, and `&`). This ensures the browser treats the input as inert text rather than executable HTML code. * **Context-Specific Encoding:** Apply encoding appropriate to the output context (HTML, JavaScript, CSS, or URL). Since this is rendered in an HTML field, **HTML Entity Encoding** is the primary requirement.

- **Input Validation:** Implement strict **server-side validation** to limit the characters, format, and length of input for fields like "Voucher ID." For example, if a Voucher ID should only contain alphanumeric characters, reject all other input.
- **Use Secure Templating Engines:** Utilize secure web application frameworks or templating engines that automatically perform output encoding by default, significantly reducing the chance of injection flaws.

10. Missing Security Headers

Severity: Medium

Description:

The application's HTTP responses are missing several standard security-focused headers like CSP, HSTS, X-XSS-Protection, and X-Content-Type-Options, which are designed to instruct the client browser on security policies. Without these headers, the browser's default, less secure settings are used, exposing users to client-side attacks.

Reproduction:

Verification of Missing Headers

- **Header Check (cURL):** Execute a non-intrusive cURL command to fetch the HTTP response headers for the target domain.
 1. **Command:** `curl -I https://www.sheba.xyz`
- **Analyze Output:** Inspect the headers returned. The output confirms the **absence** of the following crucial security headers:
 1. Strict-Transport-Security (HSTS)
 2. Content-Security-Policy (CSP)
 3. X-Frame-Options
 4. X-Content-Type-Options
 5. Referrer-Policy
 6. Permissions-Policy
- **Conclusion:** The missing headers expose the client to attacks like protocol downgrade, MIME-type sniffing, Clickjacking, and allow content sources to load arbitrarily.

Proof of CSP Absence (Inline Script Execution)

- **Target Vector Identification:** Access any user-controllable input vector on the target page that is known to be vulnerable to injection (e.g., a vulnerable URL parameter or reflected input—often the precursor to XSS).
- **Payload Submission:** Submit a benign JavaScript payload designed to prove code execution:
 1. **Payload Example:** `alert("CSP Missing - Inline JS Executed");`
- **Execution:** The payload is successfully reflected and executed by the browser, triggering an alert box (as shown in the image).
- **Verification:** The successful execution of the inline JavaScript **confirms the complete absence of a restrictive Content-Security-Policy**. A properly configured CSP would have blocked the execution of this inline script, proving the lack of this defense-in-depth mechanism.

Proof of Concept:

```

Session  Actions  Edit  View  Help
[Error@secretmind]-[~]
$ curl -I https://www.sheba.xyz
HTTP/2 200
date: Fri, 14 Nov 2025 17:25:38 GMT
content-type: text/html; charset=utf-8
content-length: 58999
etag: "e677-j0UMghknXHxcTZel2bDielFRUjA"
accept-ranges: none
vary: Accept-Encoding

```

Figure 10.1: Missing Security Headers

Security Report Summary



Site:	https://www.sheba.xyz:443/
IP Address:	13.250.208.95
Report Time:	14 Nov 2025 17:35:46 UTC
Headers:	✖ Strict-Transport-Security ✖ Content-Security-Policy ✖ X-Frame-Options ✖ X-Content-Type-Options ✖ Referrer-Policy ✖ Permissions-Policy
Advanced:	Ouch, you should work on your security posture immediately: Start Now

Missing Headers

Strict-Transport-Security	HTTP Strict Transport Security is an excellent feature to support on your site and strengthens your implementation of TLS by getting the User Agent to enforce the use of HTTPS. Recommended value "Strict-Transport-Security: max-age=31536000; includeSubDomains".
Content-Security-Policy	Content Security Policy is an effective measure to protect your site from XSS attacks. By whitelisting sources of approved content, you can prevent the browser from loading malicious assets.
X-Frame-Options	X-Frame-Options tells the browser whether you want to allow your site to be framed or not. By preventing a browser from framing your site you can defend against attacks like clickjacking. Recommended value "X-Frame-Options: SAMEORIGIN".
X-Content-Type-Options	X-Content-Type-Options stops a browser from trying to MIME-sniff the content type and forces it to stick with the declared content-type. The only valid value for this header is "X-Content-Type-Options: nosniff".
Referrer-Policy	Referrer Policy is a new header that allows a site to control how much information the browser includes with navigations and referrers. It should be set by all sites.
Permissions-Policy	Permissions Policy is a new header that allows a site to control which features and APIs can be used in the browser.

Figure 10.2: Missing Security Headers

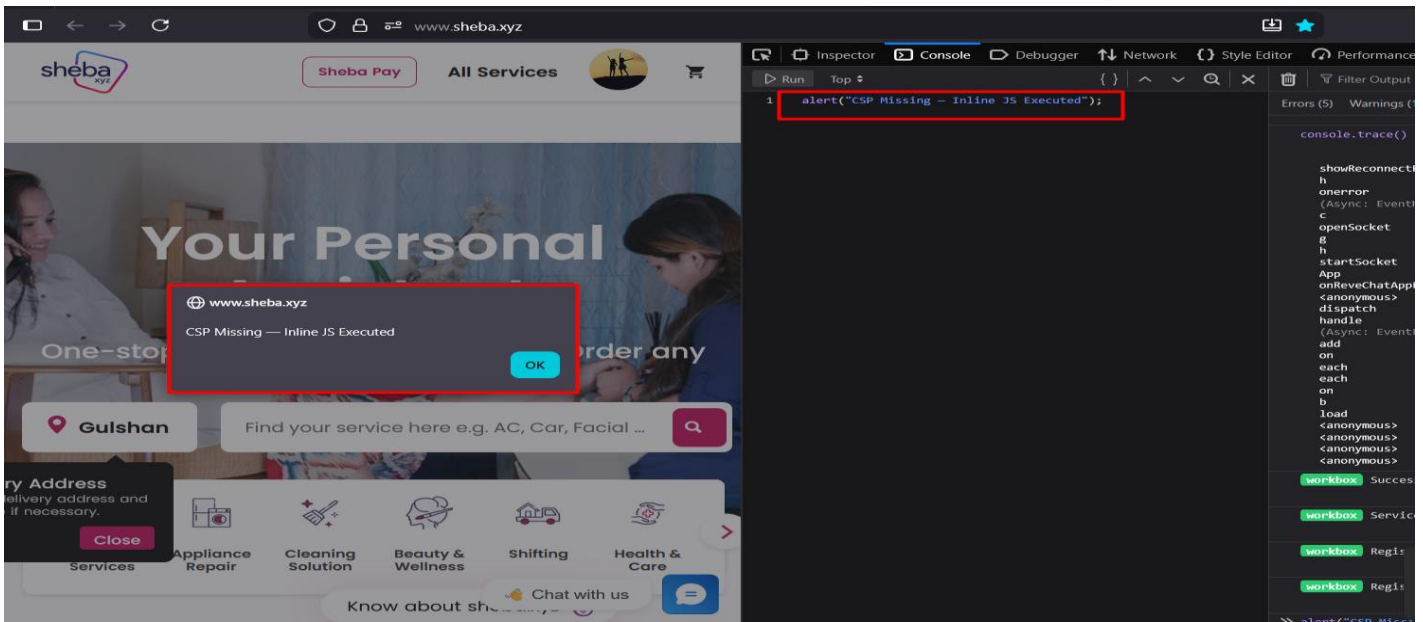


Figure 10.3: Missing Content-Security-Policy (CSP)

Impact:

The absence of these six security controls significantly increases the risk of various client-side attacks:

- **Strict-Transport-Security** - Exposes users to Protocol Downgrade Attacks (e.g., forcing a connection over insecure HTTP)
- **Content-Security-Policy** - Failure to provide a secondary defense against Cross-Site Scripting (XSS) attacks
- **X-Frame-Options** - Exposes the application to Clickjacking attacks, where users can be tricked into clicking invisible elements
- **X-Content-Type-Options** - Leaves the door open for MIME Sniffing Attacks, potentially allowing malicious files to be executed as scripts
- **Referrer-Policy** - Risk of Information Disclosure by leaking the full URL (which may contain tokens or sensitive data) to external sites
- **Permissions-Policy** - Allows use of potentially privacy-invasive browser features (geolocation, microphone, camera) without explicit control

Mitigation:

- **Content-Security-Policy (CSP):** Implement a strict policy (e.g., default-src 'self'; script-src 'self') to prevent **Cross-Site Scripting (XSS)** attacks by whitelisting trusted content sources.
- **Strict-Transport-Security (HSTS):** Implement HSTS (e.g., max-age=31536000; includeSubDomains) to enforce **HTTPS** and prevent protocol downgrade attacks.
- **X-Frame-Options:** Set this header (e.g., DENY) to prevent **Clickjacking** by restricting page embedding in frames.
- **X-Content-Type-Options:** Set to nosniff to prevent **MIME-sniffing** attacks.
- **Referrer-Policy:** Set a policy (e.g., strict-origin-when-cross-origin) to control and minimize information leakage via the Referrer header.
- **Permissions-Policy:** Implement a restrictive policy to control which browser features and APIs (e.g., camera, geolocation) are available to the page.

- **Cross-Origin Isolation Headers:** Implement the following three headers together to create an **isolated browsing context** to protect against cross-site side-channel attacks and data leakage:
 - **Cross-Origin-Embedder-Policy (COEP):** Set to require-corp.
 - **Cross-Origin-Opener-Policy (COOP):** Set to same-origin.
 - **Cross-Origin-Resource-Policy (CORP):** Set to same-origin (for internal resources).
- **Server Metadata Minimization:** Configure the web server to **remove or minimize** the **Server** and **X-Powered-By** headers to reduce information disclosure and fingerprinting risk.

11. TLS/SSL Misconfigurations

Severity: Medium

Description:

The server does not support the modern **TLS 1.3 protocol** and relies on **TLS 1.2**. Furthermore, the configuration likely supports outdated **CBC cipher suites**, which are theoretically susceptible to attacks like LUCKY13/BEAST. Failure to migrate to TLS 1.3 results in reduced cryptographic strength and lower industry security grades.

Reproduction:

Protocol Support Testing:

- **Test TLS 1.3 Support (Modern Baseline):** Execute the openssl s_client command specifically requesting the latest protocol version (-tls1_3).
 - **Command:** openssl s_client -connect sheba.xyz:443 -tls1_3
 - **Vulnerable Outcome:** The connection attempt fails or immediately closes (as shown in image image_87bb50.png), confirming the server **does not support or refuses** the most secure, modern TLS 1.3 protocol.
- **Test Weak/Obsolete TLS Versions (Downgrade Risk):** Execute the openssl s_client command explicitly requesting deprecated TLS versions (e.g., -tls1 for TLS 1.0, and -tls1_2 for the legacy current standard).
 - **Command Examples:**
 - openssl s_client -connect sheba.xyz:443 -tls1
 - openssl s_client -connect sheba.xyz:443 -tls1_2
 - **Vulnerable Outcome:** The connection successfully establishes using the weak **TLSv1 (TLS 1.0)** protocol (as shown in images image_87bb2f.png and image_87bb57.png), indicating the server is configured to accept connections over an **obsolete and insecure protocol** that is vulnerable to known attacks like **BEAST**.

Proof of Concept:

The protocol failure was confirmed using the openssl s_client utility.

```
[Error@secretmind]-[~]
$ openssl s_client -connect sheba.xyz:443 -tls1_2
Connecting to 13.251.229.42
CONNECTED(00000003)
depth=2 C=US, O=Amazon, CN=Amazon Root CA 1
verify return:1
depth=1 c=US, O=Amazon, CN=Amazon RSA 2048 M02
verify return:1
depth=0 CN=sheba.xyz
verify return:1

Certificate chain
 0 s:CN=sheba.xyz
   i:C=US, O=Amazon, CN=Amazon RSA 2048 M02
   a:PKEY: RSA, 2048 (bit); sigalg: sha256withRSAEncryption
V:NotBefore: Sep  2 00:00:00 2025 GMT; NotAfter: Oct  1 23:59:59 2026 GMT
 1 s:C=US, O=Amazon, CN=Amazon RSA 2048 M02
   i:C=US, O=Amazon, CN=Amazon Root CA 1
   a:PKEY: RSA, 2048 (bit); sigalg: sha256withRSAEncryption
V:NotBefore: Aug 23 22:25:30 2022 GMT; NotAfter: Aug 23 22:25:30 2030 GMT
 2 s:C=US, O=Amazon, CN=Amazon Root CA 1
   i:C=US, ST=Arizona, L=Scottsdale, O=Starfield Technologies, Inc., CN=Starfield Services Root Certificate Authority - G2
   a:PKEY: RSA, 2048 (bit); sigalg: sha256withRSAEncryption
V:NotBefore: May 25 12:00:00 2015 GMT; NotAfter: Dec 31 01:00:00 2037 GMT

Server certificate
-----BEGIN CERTIFICATE-----
MIIFwJCCBkgqAwIIBAGIQDLP+hBUPTUHHfmgBBF6tZANBgkqhkiG9w0BAQsFAAD
AQgCIVY0RGEzJjZjEPMGAQAUEChangWenPuhHmewegVDQDEANbmHmf624g
ULNBIDlMndgeTAYIMBAxDTI1MdkmYIAWMDawFoXoIT2MTAWtiINTKl3iOvwPFDS
MBAGA1UEAxNjczhlYnEueHlwMiBIIBjANBgkqhkiG9w0BAQEFAAOCAQMAMICBgK
AQEAu8lqjIJdgidIfjMac/tet3EIjPW6AXE5LzQSQENWTmw04o6YCTvrLKrSub06
B3nf/vqLYs1StpAB1JIJKMLK2gewcz3ooBdcXG1CN/Ep0/Y0cs/2mw0d94KZ
j0DAHESPS2scvUDLF+8osgZTj1bgstQLAKU2j6Ad3vIWgrHwhaz7BhmFn6fdvp
VT73yAqk6n5vSRbjZncPLc6Knrdh7czDcd1mVLtEgmLnA1ZzeZz/SKZ
tlZBL7HG6Axt76G0RPdwKVccPqvJHMzPd0ysapVW+cISPP/qfVSExndYldQzJA4
tL8Cwt65B3FR/wyvd9R/UemQwIDAQAoAI1C5jCCAU1wHwVDROjBBgwFoAuWDF5
zVpQwk38dHHoy+mc+XrrguIwHQVDROBBYEADVXBmLTye2t8SCH1vDZGS1+C5
NCEGA1UDEGAQMBICCXNoZWJhLnSeoILK15zaGV1YS54eXowEwVDROjBAAwCIAI
BgZngbwAgEweyVDROPAQA/BQAQAgwBMGA1UDeQMAAGCCSAQGFwBwmHos6g
A1UDeQMAAGCCSAQGFwBwmHos6gA1UDeQMAAGCCSAQGFwBwmHos6gA1UDeQMAAGCC
LS13yYtaLmlybDB1BgerBgfEBQcBAQRpmGwcLQTKYWBYBQUHMGAG1lnhDH6Ly9y
Y3NWLnlybTAyLmF1YXpvbnRydXN0LnNmTA2BgerBgfEBQcUAoYaHR0CDovL2Ny
dc5Ymm0wM1ShbfW6b250cncVzdC5jb20vczjTMdiUv2VyMMGA1UDEwEB/wQCANA
ggF/BgorBEAd25AgQBIIbbsSCAWsBaQ82AndtFRDRP/V3wsfpX9cAv/mCYTna
```

Figure 11.1: Successful Handshake Negotiation for TLS 1.2 Protocol

```
[Error@secretmind]-[~]
$ openssl s_client -connect sheba.xyz:443 -tls1
Connecting to 13.251.229.42
CONNECTED(00000003)
40F76A1D287F0000:error:0A00042E:SSL routines:ssl3_read_bytes:tlsv1 alert protocol version:../ssl/record/rec_layer_s3.c:916:SSL alert number 70
no peer certificate available
No client certificate CA names sent
SSL handshake has read 7 bytes and written 158 bytes
Verification: OK
New, (NONE), Cipher is (NONE)
Protocol: TLSv1
Secure Renegotiation IS NOT supported
Compression: NONE
Expansion: NONE
No ALPN negotiated
SSL-Session:
    Protocol : TLSv1
    Cipher : 0000
    Session-ID:
    Session-ID-ctx:
    Master-Key:
    PSK identity: None
    PSK identity hint: None
    SRP username: None
    Start Time: 1763879025
    Timeout : 7200 (sec)
    Verify return code: 0 (ok)
    Extended master secret: no
```

Figure 11.2: Failed Handshake Negotiation for TLS 1.0 Protocol

```

[Error@secretmind][-]
$ openssl s_client -connect sheba.xyz:443 -tls1_3
Connecting to 13.225.1.229:42
CONNECTED(00000003)
closed
no peer certificate available
No client certificate CA names sent
Negotiated TLS1.3 group: <NULL>
SSL handshake has read 7 bytes and written 1478 bytes
Verification: OK
New, (NONE), Cipher is (NONE)
Protocol: TLSv1.3
This TLS version forbids renegotiation.
Compression: NONE
Expansion: NONE
No ALPN negotiated
Early data was not sent
Verify return code: 0 (ok)

```

Figure 11.3: Failed Handshake Negotiation for TLS 1.3 Protocol

```
(Error@secretmind)-[~]
$ openssl s_client -connect sheba.xyz:443 -tls1 -tlsextdebug
Connecting to 3.0.167.228
CONNECTED(00000003)
406702FBE37F0000:error:0A00042E:SSL routines:ssl3_read_bytes:tls1 alert protocol version:../ssl/record/rec_layer_s3.c:916:SSL alert number 70
no peer certificate available
No client certificate CA names sent
SSL handshake has read 7 bytes and written 158 bytes
Verification: OK
New, (NONE), Cipher is (NONE)
Protocol: TLSv1
Secure Renegotiation IS NOT supported
Compression: NONE
Expansion: NONE
No ALPN negotiated
SSL-Session:
    Protocol  : TLSv1
    Cipher    : 0000
    Session-ID:
    Session-ID-context:
    Master-Key:
    PSK identity: None
    PSK identity hint: None
    SRP username: None
    Start Time: 1763878579
    Timeout   : 7200 (sec)
    Verify return code: 0 (ok)
    Extended master secret: no
```

Figure 11.4: Alternative Failed Handshake Negotiation for **TLS 1.0** Protocol

Impact:

- **Reduced Resilience:** The reliance on TLS 1.2 and the inclusion of CBC cipher suites (e.g., AES-CBC) expose the server to theoretical side-channel attacks like LUCKY13 and BEAST, which target flaws in the CBC padding mechanism.
- **Compliance Risk:** The configuration fails to meet the "**Modern**" security requirements of major compliance standards (e.g., PCI DSS, HIPAA) and vendors (e.g., Mozilla's guidelines), potentially resulting in compliance failures or lower SSL Labs security grades.
- **Reputational Damage:** Receiving a lower security grade publicly indicates that the site is not using best practices, potentially eroding user trust in the platform's ability to secure their communication.

Mitigation:

The primary goal is to shift the server from relying on the older **TLS 1.2** protocol and **weak CBC ciphers** to modern, resilient standards.

- **Enable TLS 1.3:** Immediately configure the server to **enable and prioritize TLS 1.3**. This is the current, most secure protocol.
- **Disable CBC Ciphers:** Restrict the TLS 1.2 cipher list by **disabling all CBC cipher suites** (e.g., AES256-CBC), which are vulnerable to padding oracle attacks.
- **Use GCM Ciphers:** Limit remaining TLS 1.2 support to **AEAD GCM cipher suites** (e.g., AES256-GCM), as they provide superior integrity.
- **Enforce Preference:** Enable server configuration to **honor the server's preferred cipher order** (e.g., using PFS ciphers) to prevent clients from forcing weaker connections.
- **Deploy HSTS:** Implement the **Strict-Transport-Security** HTTP header to ensure browsers always connect via HTTPS, reinforcing the TLS layer.

12.Fingerprinting & Information Disclosure

Severity: Medium

Description:

During testing, multiple subdomains of sheba.xyz were found to disclose detailed server banners, including:

- Exact web server versions (nginx, Apache)
- Operating systems (Ubuntu, CentOS)
- PHP versions (PHP/7.1.33, PHP/8.1.33)
- Frameworks (Node.js, Express, Nuxt.js, Vue.js)
- Backend technologies (Kestrel, ASP.NET)

This allows attackers to precisely fingerprint the backend infrastructure and map vulnerable component

Reproduce:

- Check Server Headers Using cURL
Command: `curl -I https://sheba.xyz`
- Enumerate Server Version Using Nmap
Command: `nmap -sV -p 80,443 sheba.xyz`
- Identify Frameworks & Technologies (httpx)
Command: `httpx -u https://www.sheba.xyz -title -tech-detect -status-code`
- Technology Detection for All Subdomains
Command: `subfinder -d sheba.xyz -silent | httpx -title -tech-detect -status-code`

Proof of Concept:

Main domain reveals AWS ELB + nginx version:

```
(Error@secretmind)-[~]
$ nmap -sV -p 80,443 sheba.xyz
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-28 03:04 +06
Nmap scan report for sheba.xyz (3.0.167.228)
Host is up (0.059s latency).
Other addresses for sheba.xyz (not scanned): 13.251.229.42 54.251.208.47
rDNS record for 3.0.167.228: ec2-3-0-167-228.ap-southeast-1.compute.amazonaws.com

PORT      STATE SERVICE VERSION
80/tcp    open  http   AWS Elastic Load Balancing
443/tcp    open  ssl/http nginx 1.18.0 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 14.27 seconds
```

Figure 12.1: Main domain reveals AWS ELB + nginx version

Subdomains leak outdated web server versions

```
Error@secretmind: ~  
Error@secretmind: ~  
(Error@ secretmind)-[~]  
$ subfinder -d sheba.xyz -silent | httpx -title -tech-detect -status-code  
  
projectdiscovery.io  
  
[INF] Current httpx version v1.7.2 (latest)  
[WRN] UI Dashboard is disabled, Use -dashboard option to enable  
http://mail.sheba.xyz [403] [Apache HTTP Server Test Page powered by CentOS] [Apache HTTP Server:2.4.6,Bootstrap,CentOS,PHP:7.1.33]  
https://bondhu.sheba.xyz [503] [503 Service Unavailable] [Apache HTTP Server:2.4.18,Ubuntu]  
https://tech.sheba.xyz [404]  
https://accountkit.sheba.xyz [404]  
https://teleport.sheba.xyz [302] [HSTS]  
https://pulse.sheba.xyz [200] [Ticketing] [Nginx:1.21.1]  
https://sso.sheba.xyz [302] [HSTS,Nginx:1.24.0,Ubuntu]  
https://api-supplier.sheba.xyz [404] [Express,Node.js]  
https://api.pulse.sheba.xyz [200] [Sheba Business Portal] [Nginx:1.21.1]  
https://sentry.sheba.xyz [302] [Nginx]  
https://accounts.sheba.xyz [302] [Redirecting to https://accounts.sheba.xyz/login] [Nginx:1.25.5,PHP:8.1.33]  
https://supplier.sheba.xyz [200] [Dealers Management System] [Nginx:1.21.1]  
https://business.sheba.xyz [200] [sBusiness.xyz | Your Business Assistant] [Amazon S3,Amazon Web Services,Apache HTTP Server:2.4.18,Ubuntu]  
https://api.logistics.sheba.xyz [200] [Apache HTTP Server:2.4.18,Ubuntu]  
https://admin.logistics.sheba.xyz [200] [Be right back.] [Apache HTTP Server:2.4.18,Ubuntu]  
https://www.sheba.xyz [200] [Get Expert Professional Services at Home in Bangladesh | Sheba.xyz] [Google Tag Manager,Node.js,Nuxt.js,SweetAlert2:11,Tailwind CSS,Vue.js,jsDelivr]  
https://admin.sheba.xyz [404] [Whoops.] [Apache HTTP Server:2.4.18,Ubuntu]  
http://cd.edotco.sheba.xyz [200] [Your Azure Function App is up and running.] [Kestrel,Microsoft ASP.NET,Microsoft Ajax Content Delivery Network,jQuery UI:1.13.2,jQuery:3.6.0]
```

Figure 12.2: Subdomains leak outdated web server versions

Findings from httpx -tech-detect:

- http://mail.sheba.xyz [Apache 2.4.6, PHP 7.1.33, CentOS]
- https://business.sheba.xyz [Apache 2.4.18, Ubuntu]
- https://pulse.sheba.xyz [Nginx 1.21.1]
- https://accounts.sheba.xyz [Nginx 1.25.5, PHP 8.1.33]
- https://api-supplier.sheba.xyz [Express, Node.js]
- http://cd.edotco.sheba.xyz [Kestrel, ASP.NET]

Framework disclosure:

- https://www.sheba.xyz [Nuxt.js, Vue.js, Tailwind CSS]

Impact

Attackers can map the entire technology stack and target known vulnerabilities, including:

- Outdated Apache versions (2.4.6 / 2.4.18) → RCE & Path Traversal
- Outdated nginx versions (1.18.0 / 1.21.1) → Memory corruption CVEs
- PHP 7.1.33 → End-of-life & vulnerable
- Node.js Express apps → Prone to misconfigurations
- OS fingerprinting → Easier targeted exploitation

Effective penetration often begins from information gathered during fingerprinting.

This significantly increases risk of:

- Targeted exploits
- Privilege escalation
- Automated attacks
- Infrastructure mapping
- Reconnaissance by threat actors

Mitigation:

- Disable or minimize version banners in all web servers (Apache, Nginx).
- Remove X-Powered-By, Server, and framework-leaking headers.
- Use a reverse proxy to mask backend technologies.
- Update vulnerable services (Apache 2.4.6 → extremely outdated).
- Implement WAF / CDN headers obfuscation.
- Periodically scan infrastructure for banner leaks.

Recommendations (Global Fixes)

- Implement strict access control checks on all API endpoints to prevent IDOR vulnerabilities and ensure users can only access their own data.
- Enforce server-side validation for all MFA and OTP-related parameters, ensuring that values such as retry timers and phone numbers cannot be manipulated in transit.
- Strengthen business logic controls to prevent unauthorized price changes, negative quantity manipulation, and other cart-related abuses.
- Enable Secure and HttpOnly flags on all sensitive cookies to reduce the risk of theft or client-side access.
- Add missing security headers such as X-Frame-Options, Content-Security-Policy, X-Content-Type-Options, and Referrer-Policy to prevent clickjacking, XSS, and content injection attacks.
- Fix TLS/SSL configuration issues by removing weak ciphers, enabling modern protocols, and ensuring certificates remain valid and properly configured.
- Implement SPF, DKIM, and DMARC correctly to prevent email spoofing and strengthen email authentication.
- Add proper rate limiting, lockout mechanisms, and brute-force protections for login, OTP, and password recovery endpoints.
- Improve input validation and sanitization to eliminate Self-XSS, Text Injection, and similar client-side vulnerabilities.
- Update outdated software, remove unnecessary services, and disable fingerprinting banners to reduce information disclosure.
- Restrict access to non-sensitive files and directories and remove unnecessary public resources such as .txt files, config files, debug info, and exposed endpoints.
- Introduce regular security testing, code reviews, and monitoring to detect anomalies, unauthorized changes, and misuse of APIs.

Conclusion

The assessment of the target application revealed several security weaknesses across authentication, authorization, input handling, and overall application configuration. Critical issues such as MFA OTP manipulation, OTP redirection, and IDOR demonstrate a lack of proper access control and validation, allowing attackers to bypass security mechanisms and access sensitive customer information. Additional findings—including business logic flaws, insecure cookies, missing security headers, email spoofing weaknesses, clickjacking, and TLS/SSL shortcomings—further indicate gaps in secure development practices and defensive hardening.

If exploited, these vulnerabilities could significantly impact user privacy, data integrity, and overall system security. The findings highlight the need for immediate remediation, implementation of secure coding standards, stronger validation controls, and improved monitoring. Addressing these issues will help ensure the security and reliability of the platform while reducing the risk of future exploitation.